REDIS FOR AI ENGINEERS · ROADMAP HANDBOOK

04

System Design with Redis

The phase where Redis stops being three commands (**SET**, **GET**, **DEL**) and starts being an architecture decision. This handbook covers distributed caching, invalidation, stampede control, replication, Sentinel, Cluster, and a full capstone project — everything you need to talk about Redis at an interview-level, system-design depth.

Distributed Cache | Cache Invalidation | Cache Stampede | Cache Warming | Replication | Sentinel | Cluster | Capstone Project

---

YOUR ROADMAP: AI ENGINEER → SENIOR AI ENGINEER → AI ARCHITECT

Caching → Redis Fundamentals → SWE Use Cases → **System Design** → AI/ML Use Cases → Agentic AI → Production

CONTENTS

Phase 4 Handbook

01 **Distributed Caching**
One Redis brain shared by many app servers — connection pooling done right.

02 **Cache Invalidation**
The hardest problem in caching — explicit invalidation, versioned keys, and events.

03 **Cache Stampede**
Why a single expiring key can crash a database — and two ways to stop it.

04 **Cache Warming**
Preloading the cache before real traffic ever hits it.

05 **Redis Replication**
Master-replica architecture — scale reads, protect writes.

06 **Redis Sentinel**
Automatic failover — Redis watching Redis, so your app doesn't have to.

07 **Redis Cluster**
Sharding across 16,384 hash slots to scale writes horizontally.

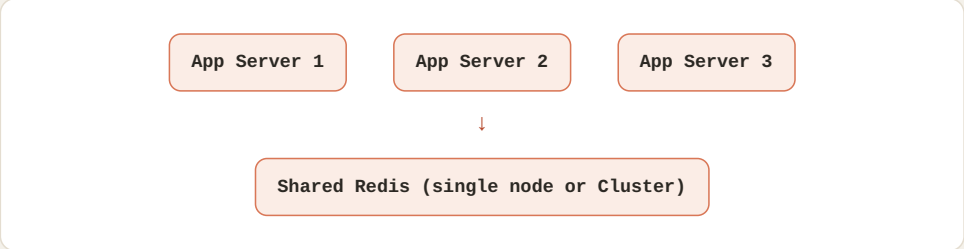
08 **Capstone: High-Traffic API**
Project 3, end to end — NGINX, FastAPI, Redis, and Postgres working together.

✓ **Cheat Sheet**
Every concept in this phase, on one page.

CHAPTER 01

Distributed Caching

Once you have more than one app server, Redis stops being "your cache" and becomes **shared infrastructure**. Every instance of your API — App 1, App 2, App 3 — talks to the exact same Redis node (or the same Cluster), so a value cached by one request is instantly visible to a request served by a completely different server.



This is what makes horizontal scaling actually work: you can add a fourth, fifth, or hundredth app server behind a load balancer, and they all share one consistent cache layer instead of each keeping its own private, out-of-sync copy of the data.

Connection pooling

Opening a brand-new TCP connection to Redis on every single request is one of the most common beginner mistakes — it adds latency and can exhaust Redis's max client limit under load. Instead, each app process creates **one pool** at startup and reuses connections from it.

DISTRIBUTED_CACHE.PY

```
import redis

# DISTRIBUTED CACHE: many app servers share ONE Redis (or one Redis Cluster).
# The connection POOL is created once per app process and reused for every
# request — never open a fresh TCP connection per request, it kills throughput.
redis_pool = redis.ConnectionPool(
    host="redis-server",     # could be a single node or a Cluster endpoint
    port=6379,
    db=0,
    max_connections=50,      # cap so one app instance can't starve the others
    decode_responses=True
)

def get_redis_client():
    # App1, App2, App3 ... every instance calls this and all of them
    # end up talking to the SAME Redis brain.
    return redis.Redis(connection_pool=redis_pool)

r = get_redis_client()

def get_product(product_id: str):
    cache_key = f"product:{product_id}"
    cached = r.get(cache_key)
    if cached:
        return cached                       # CACHE HIT: no DB call at all

    data = query_product_from_db(product_id)   # CACHE MISS: go to source of truth
    r.setex(cache_key, 300, data)               # store with a 5 minute TTL
    return data
```

COMMON MISTAKE

Creating `redis.Redis(host=..., port=...)` fresh inside every request handler. Under real traffic this creates thousands of short-lived connections per second and Redis will start rejecting new ones. Always share one pool per process.

CHAPTER 02

Cache Invalidation

There's an old engineering joke: *"There are only two hard things in Computer Science: cache invalidation and naming things."* Redis makes storing data trivial — TTL makes **eventual** correctness trivial too. The hard part is deciding **when** a cached value must be thrown away immediately, before its TTL naturally expires, because the underlying data already changed.

Strategy	How it works	Best for
Explicit invalidation	Delete the exact cache key right after the DB write.	Simple CRUD APIs, single-key updates.
Versioned keys	Bump a version counter; old keys become orphaned and expire naturally.	Bulk changes affecting many related keys at once.
Event-driven (Pub/Sub)	Publish a "this key changed" event; every service listening deletes its local copy.	Multiple services/microservices sharing one cache.

CACHE_INVALIDATION.PY

```
# STRATEGY 1 — Explicit (active) invalidation
# The moment the DB changes, delete the now-stale cache entry.
def update_product_price(product_id: str, new_price: float):
    update_product_in_db(product_id, new_price)   # 1. update source of truth
    r.delete(f"product:{product_id}")             # 2. drop the stale copy
    # next get_product() call is a guaranteed MISS -> fetches fresh data


# STRATEGY 2 — Versioned keys
# Instead of hunting down every related key, bump a version number.
# All old keys become orphaned and simply expire via TTL.
def get_catalog(category: str):
    version = r.get("catalog:version") or "1"
    cache_key = f"catalog:{category}:v{version}"   # version baked into the key
    cached = r.get(cache_key)
    if cached:
        return cached
    data = query_catalog_from_db(category)
    r.setex(cache_key, 3600, data)
    return data

def invalidate_entire_catalog():
    r.incr("catalog:version")   # one INCR invalidates every catalog:* key at once


# STRATEGY 3 — Event-driven invalidation (Pub/Sub)
# Useful when several services share one cache and must be told in real time.
def publish_invalidation_event(key: str):
    r.publish("cache-invalidation-channel", key)

def cache_invalidation_listener():
    pubsub = r.pubsub()
    pubsub.subscribe("cache-invalidation-channel")
    for message in pubsub.listen():
        if message["type"] == "message":
            stale_key = message["data"]
            r.delete(stale_key)   # remove the key this OTHER service changed
```

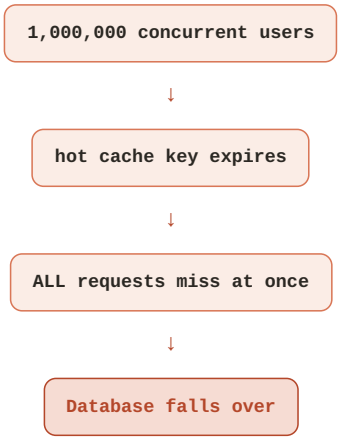
KEY IDEA

TTL is your **safety net**, not your invalidation strategy. Always set a TTL even when you invalidate explicitly — it guarantees that if an invalidation event is ever missed, the stale data self-heals within a bounded time window.

CHAPTER 03

Cache Stampede

Also called the **thundering herd problem**. It happens when a single, very "hot" cache key expires while traffic is high: every one of those concurrent requests misses the cache at the same moment and all of them fall through to the database together.



Two well-known techniques prevent this from ever reaching the database:

1. Distributed lock (mutex)

Only the first request to notice the miss is allowed to rebuild the cache. Every other concurrent request either waits briefly and retries, or (in a more advanced version) reads the previous, slightly-stale value while the rebuild is in progress.

2. Probabilistic early expiration

Instead of a hard cliff-edge expiry, each request close to the TTL boundary has a small, randomized chance of triggering an early refresh. This spreads rebuilds out smoothly over time instead of letting them all collide at the exact expiry instant.

CACHE_STAMPEDE.PY

```python
import time, uuid, random, math

# THE PROBLEM
# A hot key expires -> thousands of concurrent requests miss the cache
# AT THE SAME INSTANT -> all of them hit the DB together -> DB falls over.

# SOLUTION 1 — Distributed lock (mutex)
# Only ONE request is allowed to rebuild the cache; everyone else waits.
def get_with_lock(product_id: str):
    cache_key = f"product:{product_id}"
    lock_key = f"lock:{cache_key}"

    cached = r.get(cache_key)
    if cached:
        return cached

    lock_id = str(uuid.uuid4())
    # NX = only set if the lock doesn't already exist, PX = auto-expire in ms
    got_lock = r.set(lock_key, lock_id, nx=True, px=5000)

    if got_lock:
        try:
            data = query_product_from_db(product_id)   # only WE do this
            r.setex(cache_key, 300, data)
            return data
        finally:
            r.delete(lock_key)          # release the lock when done
    else:
        time.sleep(0.05)                # someone else is already rebuilding
        return get_with_lock(product_id)   # brief retry


# SOLUTION 2 — Probabilistic early expiration (the "xfetch" idea)
# Refresh slightly BEFORE hard expiry, and only for a small % of requests,
# so the rebuild happens smoothly instead of all at once.
def get_with_early_refresh(product_id: str):
    cache_key = f"product:{product_id}"
    cached = r.get(cache_key)
    ttl = r.ttl(cache_key)          # seconds remaining before expiry
    beta = 1.0                      # tuning knob: higher = refresh earlier
    original_ttl = 300

    if cached and ttl > 0:
        should_refresh_early = (original_ttl * beta * math.log(random.random())) < -ttl
        if not should_refresh_early:
            return cached
        # else: fall through and refresh now, slightly early, on purpose

    data = query_product_from_db(product_id)
    r.setex(cache_key, original_ttl, data)
    return data
```

> **INTERVIEW TIP**
>
> If you're asked "how would you protect a database from a cache stampede", the lock-based answer is the one interviewers expect first — it's simpler to reason about and to explain on a whiteboard. Mention probabilistic early expiration as the more advanced follow-up.

CHAPTER 04

Cache Warming

A freshly deployed service starts with a completely empty cache. If it goes live under real load immediately, the very first wave of users causes a stampede of cache misses — exactly the situation Chapter 3 warned about. **Cache warming** avoids this by populating the most important keys *before* traffic arrives.

- **On deploy** — preload the top N most-requested items right after startup.
- **On a schedule** — refresh popular keys nightly, e.g. at 3 AM low-traffic hours.
- **Before a known spike** — a flash sale or product launch, warm the cache minutes ahead.

CACHE_WARMING.PY

```
# CACHE WARMING
# Preload frequently requested data BEFORE real traffic arrives —
# typically right after a deploy, or on a schedule (e.g. before a sale).
def warm_cache_on_startup():
    top_products = get_top_100_products_from_db()   # e.g. most viewed, last 7 days
    pipeline = r.pipeline()   # batch every SET into a single network round-trip

    for product in top_products:
        cache_key = f"product:{product['id']}"
        pipeline.setex(cache_key, 3600, product["data"])

    pipeline.execute()
    print(f"Warmed {len(top_products)} products into cache")

# Typically wired into the app's startup hook, e.g. in FastAPI:
# @app.on_event("startup")
# async def startup_event():
#     warm_cache_on_startup()
```

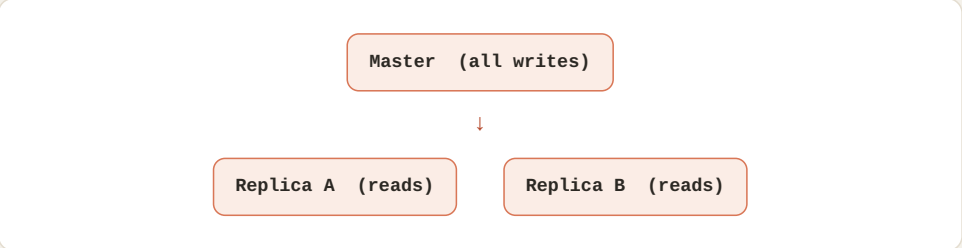
KEY IDEA

Always use a **pipeline** when warming many keys. A pipeline batches every command into one network round-trip instead of one round-trip per key — warming 100 keys individually can take seconds; pipelined, it takes milliseconds.

CHAPTER 05

Redis Replication

Replication copies data from one **master** node to one or more **replica** nodes in near real time. The master is the only node that accepts writes; replicas keep an up-to-date copy and can serve read traffic, spreading load across multiple machines.



REDIS + REPLICA .CONF

```
# redis-master.conf
port 6379
appendonly yes

# redis-replica.conf
port 6380
replicaof redis-master-host 6379   # this node replicates FROM the master
```

REPLICATION_CLIENT.PY

```
import redis

# REDIS REPLICATION — write to MASTER, read from REPLICA.
# This spreads read traffic across replicas so the master stays free for writes.
master = redis.Redis(host="redis-master", port=6379, decode_responses=True)
replica = redis.Redis(host="redis-replica", port=6380, decode_responses=True)

def write_data(key, value):
    master.set(key, value)      # ALL writes go to the master, always

def read_data(key):
    value = replica.get(key)    # reads are offloaded to the replica
    if value is None:
        # rare edge case: replica hasn't caught up yet (replication lag)
        value = master.get(key)
    return value
```

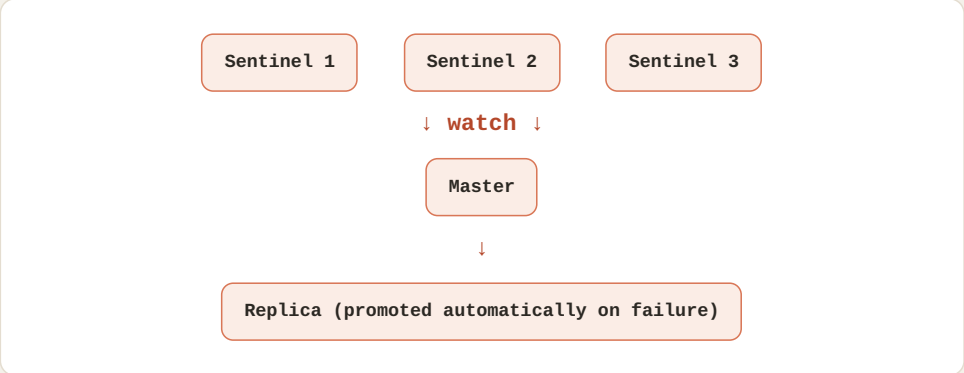
WATCH OUT FOR

Replication lag. Replicas apply changes asynchronously by default, so a replica can briefly lag a few milliseconds (or, under heavy load, much longer) behind the master. If a user writes data and immediately reads it back from a replica, they might not see their own write yet. For "read-your-own-write" consistency, read from the master right after a write.

CHAPTER 06

Redis Sentinel

Replication alone doesn't handle master failure automatically — if the master dies, someone has to promote a replica manually. **Sentinel** is Redis's own high-availability watchdog: a small cluster of Sentinel processes constantly pings the master, and if enough of them agree it's down, they elect and promote a replica as the new master — automatically.



SENTINEL.CONF

```
# sentinel.conf
sentinel monitor mymaster redis-master-host 6379 2
# "2" = quorum: at least 2 sentinels must agree the master is down
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 10000
```

SENTINEL_CLIENT.PY

```
from redis.sentinel import Sentinel

# REDIS SENTINEL — automatic failover if the master dies.
# Sentinels constantly watch the master; if it goes down, they elect
# and promote a replica to be the new master, automatically.
sentinel = Sentinel(
    [("sentinel1", 26379), ("sentinel2", 26379), ("sentinel3", 26379)],
    socket_timeout=0.5
)

# discover_master() asks the sentinels "who is master right now?" — your
# app never hardcodes a host, so failover is invisible to application code.
master = sentinel.master_for("mymaster", socket_timeout=0.5)
replica = sentinel.slave_for("mymaster", socket_timeout=0.5)

master.set("order:1001", "paid")
print(replica.get("order:1001"))
```

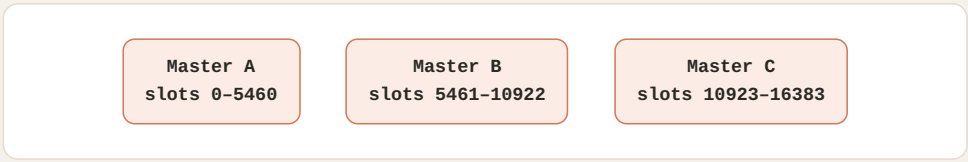
KEY IDEA

The **quorum** is the minimum number of Sentinels that must agree the master is unreachable before a failover is triggered. This prevents a single Sentinel's own network hiccup from causing an unnecessary, disruptive failover.

CHAPTER 07

Redis Cluster

Replication and Sentinel solve **availability** — but every write still goes through a single master. **Redis Cluster** solves **scale**: data is split across multiple masters using 16,384 hash slots. Each key is hashed to exactly one slot, and that slot lives on one specific master node — so writes themselves scale horizontally, not just reads.



SHELL

```
# Create a cluster with 3 masters + 3 replicas (6 nodes = real HA minimum)
redis-cli --cluster create \
  127.0.0.1:7000 127.0.0.1:7001 127.0.0.1:7002 \
  127.0.0.1:7003 127.0.0.1:7004 127.0.0.1:7005 \
  --cluster-replicas 1
```

CLUSTER_CLIENT.PY

```
from redis.cluster import RedisCluster

# REDIS CLUSTER — data is SHARDED across masters using 16384 hash slots.
# Every key hashes to one slot, and that slot lives on one specific master.
# Unlike replication, this scales WRITES horizontally too.
rc = RedisCluster(host="127.0.0.1", port=7000, decode_responses=True)

rc.set("user:1", "Alice")   # client auto-routes to the correct node
rc.set("user:2", "Bob")     # might land on a totally different node

print(rc.get("user:1"))

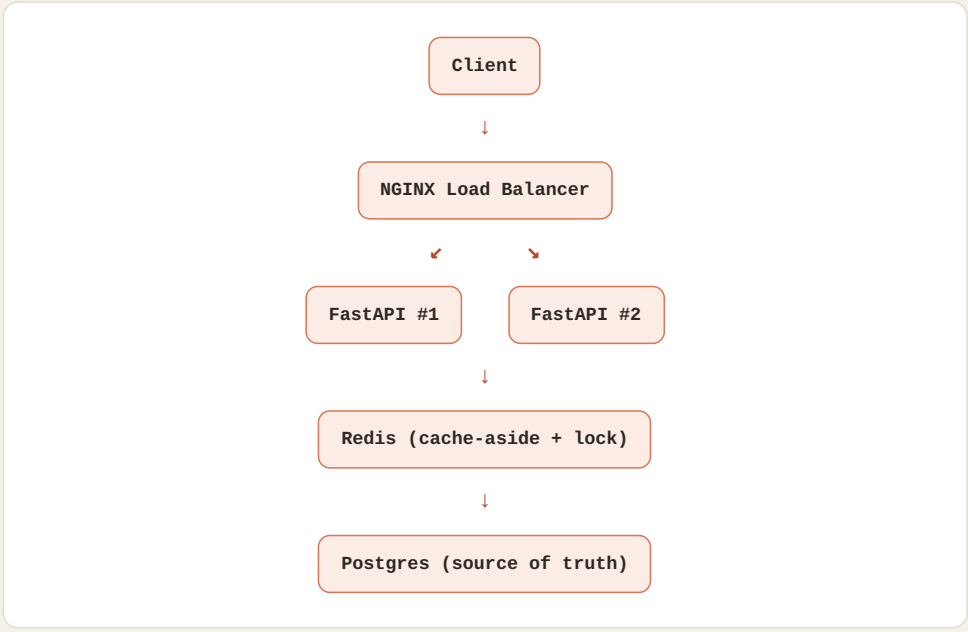
# Use {hashtags} to force related keys onto the SAME slot/node —
# needed if you want to pipeline or MULTI/EXEC across them together.
rc.set("cart:{user:1}:items", "sword,shield")
rc.set("cart:{user:1}:total", "150")
```

KEY IDEA

Multi-key operations (pipelines, transactions) only work if all involved keys live on the **same slot**. Use a `{hashtag}` — the part inside curly braces — to force related keys onto one node, as shown in the `cart:{user:1}:*` example above.

Capstone: High-Traffic API

This is **Project 3** from your roadmap, built end to end. It combines every concept from this phase into one working system: a load balancer distributing traffic across multiple FastAPI instances, all sharing one Redis cache (Chapter 1), with stampede protection (Chapter 3), cache warming on boot (Chapter 4), and explicit invalidation on writes (Chapter 2) — backed by Postgres as the permanent source of truth.



docker-compose.yml

Spins up the whole stack: NGINX, two FastAPI replicas, Redis, and Postgres.

DOCKER-COMPOSE.YML

```
version: "3.9"

services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro   # load balancer config
    depends_on:
      - api1
      - api2

  # Two identical FastAPI instances behind the load balancer —
  # this horizontal scaling is what makes it "high traffic" ready.
  api1:
    build: .
    environment:
      - INSTANCE_NAME=api1
    depends_on: [redis, postgres]

  api2:
    build: .
    environment:
      - INSTANCE_NAME=api2
    depends_on: [redis, postgres]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: appdb
    ports: ["5432:5432"]
```

nginx.conf

The load balancer — round-robins requests across `api1` and `api2`.

NGINX.CONF

```
events {}

http {
    # round-robins between the two FastAPI instances by default
    upstream fastapi_backend {
        server api1:8000;
        server api2:8000;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://fastapi_backend;
            proxy_set_header Host $host;
        }
    }
}
```

main.py

The FastAPI application. Read it top to bottom — every Phase 4 concept is labeled inline.

MAIN.PY

```
import time
import redis
import asyncpg
from fastapi import FastAPI

app = FastAPI()

# One Redis pool per app instance (api1, api2, ...). All instances point
# at the SAME Redis -> this is "Distributed Cache" from this chapter, live.
redis_pool = redis.ConnectionPool(host="redis", port=6379, decode_responses=True,
max_connections=50)
r = redis.Redis(connection_pool=redis_pool)

db_pool = None
CACHE_TTL = 60   # seconds


@app.on_event("startup")
async def startup():
    global db_pool
    db_pool = await asyncpg.create_pool(user="app", password="app", database="appdb",
host="postgres")
    await warm_cache()   # Cache Warming: run once at boot


async def warm_cache():
    """Preload popular products so the FIRST real requests are already hits."""
    async with db_pool.acquire() as conn:
        rows = await conn.fetch("SELECT id, name, price FROM products ORDER BY views DESC
LIMIT 50")
    pipeline = r.pipeline()
    for row in rows:
        pipeline.setex(f"product:{row['id']}", CACHE_TTL, f"{row['name']}:
{row['price']}")
    pipeline.execute()


@app.get("/products/{product_id}")
async def get_product(product_id: int):
    cache_key = f"product:{product_id}"

    cached = r.get(cache_key)                # 1. Cache-Aside: try cache first
    if cached:
        return {"source": "cache", "data": cached}

    lock_key = f"lock:{cache_key}"           # 2. Stampede protection
    got_lock = r.set(lock_key, "1", nx=True, px=3000)
    if not got_lock:
        time.sleep(0.05)
        return await get_product(product_id)   # someone else is rebuilding — retry

    try:
        async with db_pool.acquire() as conn:
            row = await conn.fetchrow("SELECT name, price FROM products WHERE id = $1",
product_id)
        if not row:
            return {"error": "not found"}

        data = f"{row['name']}:{row['price']}"
        r.setex(cache_key, CACHE_TTL, data)   # 3. populate cache for next request
        return {"source": "db", "data": data}
    finally:
        r.delete(lock_key)


@app.put("/products/{product_id}")
async def update_product(product_id: int, price: float):
    async with db_pool.acquire() as conn:
        await conn.execute("UPDATE products SET price = $1 WHERE id = $2", price,
product_id)

    r.delete(f"product:{product_id}")   # Cache Invalidation: DB changed, drop stale copy
    return {"status": "updated"}
```

requirements.txt

REQUIREMENTS.TXT

```
fastapi
uvicorn[standard]
redis
asyncpg
```

> **TRY THIS**
>
> Run `docker compose up --build`, then hit `GET /products/1` twice — the first response should say `"source": "db"`, the second `"source": "cache"`. Then call `PUT /products/1` and repeat the GET — it should fall back to `"db"` again, proving invalidation worked.

QUICK REFERENCE

Phase 4 Cheat Sheet

Concept	Problem it solves	Core mechanism
Distributed Cache	Multiple app servers need one consistent cache	Shared Redis + connection pooling
Cache Invalidation	Stale data outliving a DB change	Explicit delete / versioned keys / pub-sub events
Cache Stampede	Many requests miss a hot key at once	Distributed lock / probabilistic early refresh
Cache Warming	Empty cache at deploy time or before a spike	Pipelined preload on startup / schedule
Replication	Scaling reads, redundancy	One master, N replicas, async copy
Sentinel	Automatic failover on master death	Quorum-based monitoring + auto-promotion
Cluster	Scaling writes, huge datasets	16,384 hash slots sharded across masters

HOW THESE FIT TOGETHER IN PRODUCTION

A real production Redis setup is usually **Cluster + Replication + Sentinel-style failover built in** (Redis Cluster ships its own failover) — sharded for scale, replicated for safety, with the invalidation and stampede patterns from Chapters 2–3 applied on top by your application code.

Next

On to Phase 5: AI/ML Redis

You now understand Redis the way a systems engineer does — distributed, invalidated correctly, stampede-safe, replicated, and cluster-ready. Phase 5 builds directly on top of this: LLM response caching, embedding caches, and RAG retrieval caching all reuse exactly the patterns in this handbook.